

Part 1: Tell Me About Yourself & HR (8 Questions)

1. Tell me about yourself.

Answer: > "I am a Software Developer with around 2 years of professional experience specializing in the backend .NET ecosystem and SQL Server. Over the past two years, I've focused heavily on building and optimizing web applications using C#, ASP.NET Core Web API, and Entity Framework Core.

In my day-to-day work, I handle everything from designing database schemas and writing complex stored procedures to building secure, scalable RESTful APIs. For example, I recently optimized a reporting engine by switching heavy Excel generation logic over to MiniExcel, which resolved major system timeout issues during large data downloads. I'm comfortable working across the full development lifecycle, managing code with Git/GitLab, and collaborating closely with team leads and stakeholders to deliver clean, maintainable code. I'm looking to take on my next challenge where I can deepen my architectural skills and contribute to high-performance backend systems."

2. What are your core strengths and weaknesses?

- **Strengths:** "My strongest skill is my database optimization and problem-solving ability in backend logic. I don't just write code that works; I focus on execution speed, query profiles, and memory footprint. I am also highly adaptable and able to pick up new libraries or frameworks rapidly when a project demands it."
- **Weaknesses:** "Early on, I tended to dive straight into coding the moment I received a requirement. I realized this sometimes led to refactoring work later if an edge case was missed. I've corrected this by spending more upfront time analyzing the database schema, mapping out edge cases, and discussing the workflow with my lead before writing the first line of code."

3. Why are you looking for a job change?

Answer: > "I have had a fantastic foundation over the last two years, mastering the fundamentals of .NET Core, SQL optimization, and API design. However, I feel I am now ready to step out of my comfort zone. I am looking for a role where I can work on higher-scale applications, dive deeper into cloud architecture (like Azure), microservices, and system design patterns, while contributing to a larger engineering team."

4. How do you handle pressure or tight production deadlines?

Answer: > "I handle pressure by prioritizing tasks systematically and maintaining transparent communication. When a deadline is tight, I break down the remaining deliverables into a granular checklist. If I foresee a bottleneck—such as a complex third-party integration or an optimization issue—I flag it to my manager or lead early with proposed solutions, rather than

waiting until the final deadline day. This ensures the team can realign or descop non-critical items if necessary."

5. Describe a situation where you had a conflict with a teammate or manager and how you resolved it.

Answer: > "In a recent project, we had a disagreement regarding how to handle an attendance summary report column (AgendaName). The system was recording conflicting entries like 'Market Visit' and 'End Day' simultaneously. The team lead initially preferred a quick system log fix, while I felt a defined priority hierarchy based on actual business activity was necessary to avoid data corruption in reports.

Instead of arguing, I pulled sample database records, mapped out how the conflicting data looked downstream in reporting, and presented a clear priority logic. By showing the technical impact objectively, my lead agreed with my approach, and we implemented the hierarchy smoothly."

6. Where do you see yourself in the next 3 to 5 years?

Answer: > "In the next 3 to 5 years, I see myself transitioning into a Senior Software Engineer or Technical Architect role. I want to deep-dive into enterprise system design, distributed systems, and cloud-native architecture. My goal is to become the go-to person for solving complex architectural bottlenecks and mentoring junior developers, while staying closely aligned with the evolving .NET ecosystem."

7. How do you keep yourself updated with the latest technologies?

Answer: > "I regularly follow official channels like the Microsoft .NET Blog and GitHub releases to track upcoming features in new .NET versions. I also follow prominent tech content creators and community contributors on LinkedIn, read technical blogs on Medium or dev.to, and actively experiment by building small proof-of-concept projects when new features like minimal APIs or performance updates are introduced."

8. What is your understanding of the role you have applied for?

Answer: > "Based on our discussion and the job description, this is a Software Developer role focused on building high-performance, scalable backend systems using ASP.NET Core, C#, and SQL Server. It requires someone who can independently own features, design clean database schemas, write optimized queries, build secure APIs, and collaborate effectively using Git. I understand it demands someone who balances speed with code quality and clean architecture principles."

Part 2: C# Fundamentals (10 Questions)

1. What is the difference between Value Types and Reference Types? Where are they stored?

- **Value Types:** Directly contain their data. Examples include primitives (int, bool, char, double) and struct. They are typically stored on the **Stack** (or inline inside objects on the heap).
- **Reference Types:** Store a reference (pointer) to the actual data. Examples include class, interface, string, delegate, and object. The actual data object is stored on the **Heap**, while the pointer variable sits on the the **Stack**.

// Value Type Example

```
int x = 10;
int y = x; // Copy of value is made. Changing y does not affect x.
y = 20;
```

// Reference Type Example

```
class Person { public string Name { get; set; } }
Person p1 = new Person { Name = "Narendra" };
Person p2 = p1; // Copy of the reference pointer is made. Both point to the same object.
p2.Name = "Singh"; // Modifying p2 changes p1.Name as well!
```

2. Explain the difference between string, StringBuilder, and why strings are immutable.

- **String Immutability:** Once a string object is created in memory, it cannot be modified. Every modification (like concatenation) creates a brand-new string object in the managed heap, destroying or abandoning the old one. This can cause severe memory fragmentation and GC overhead in large loops.
- **StringBuilder:** Modifiable (mutable) buffer of characters. It modifies the underlying character array directly without allocating new objects every time.

```
using System.Text;
```

// Bad practice for loops: Creates 10,000 string objects in memory

```

string result = "";
for (int i = 0; i < 10000; i++) {
    result += i.ToString();
}

// Best practice: Allocates a single buffer and resizes efficiently
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.Append(i);
}
string finalResult = sb.ToString();

```

3. What is Boxing and Unboxing? What are their performance implications?

- **Boxing:** The process of converting a value type to a reference type (object). The runtime allocates memory on the heap and copies the value type's value into it.
- **Unboxing:** The explicit process of converting a boxed reference type back to a value type.
- **Performance:** Both operations are highly CPU-intensive and cause garbage collection pressure because boxing involves dynamic heap allocations.

```

int age = 25;           // Value type on stack
object boxedAge = age; // Boxing: Allocated on Heap!

int unboxedAge = (int)boxedAge; // Unboxing: Explicit cast back to stack

```

4. Explain the difference between ref, out, and in parameters.

- **ref:** The variable must be initialized *before* being passed to the method. The method can read and modify it.
- **out:** The variable does *not* need to be initialized before passing. The method *must* assign a value to it before returning.
- **in:** Introduced in modern C#; passes arguments by reference but makes them **read-only** inside the called method, preventing modification. Useful for optimizing large structs.

```

public class ParameterDemo
{
    public void Execute
    {
        int a = 10;
        ModifyRef(ref a); // 'a' must be initialized

        int b;
        InitializeOut(out b); // 'b' initialized inside method

        int c = 100;
        ReadOnlyIn(in c); // 'c' cannot be changed inside method
    }

    private void ModifyRef ref int    => x += 5;
    private void InitializeOut out int => x = 42;
    private void ReadOnlyIn in int    { /* int val = x; x = 10; -> Compilation Error! */ }
}

```

5. What are Generics in C#? How do they provide type safety and performance advantages?

Generics allow you to write a class, interface, or method with placeholders for the data types it acts upon.

- **Type Safety:** The compiler enforces strict type checking at compile-time, preventing `InvalidCastException` at runtime.
- **Performance:** Generics eliminate the need for boxing/unboxing when working with value types inside collections.

```

// Non-Generic Collection (Bad: Causes Boxing and lacks type safety)
System.Collections.ArrayList list = new System.Collections.ArrayList();

```

```
list.Add(10); // Boxed to object
list.Add("Hello"); // Mixed types allowed accidentally
int val = (int)list[0]; // Requires unboxing cast

// Generic Collection (Best Practice)
System.Collections.Generic.List<int> genericList = new
System.Collections.Generic.List<int>();
genericList.Add(10); // No Boxing! Directly stored as int
// genericList.Add("Hello"); // Compile-time error! Prevents bugs early
int genericVal = genericList[0]; // No cast needed
```

6. What is the difference between IEnumerable, ICollection, and IQueryable?

- **IEnumerable<T>**: Read-only, forward-only iteration. Best suited for in-memory collections. When querying a database via LINQ-to-SQL, it executes the query immediately, pulls **all records** into web server memory, and filters them in-memory (**Client-side filtering**).
- **ICollection<T>**: Inherits from IEnumerable. Extends it by adding modification operations like Add(), Remove(), Clear(), and a Count property.
- **IQueryable<T>**: Inherits from IEnumerable. Best suited for querying out-of-memory data sources (like databases). It generates an Expression Tree, evaluating filters lazily so that the final executed query contains the WHERE clauses (**Server-side filtering**).

```
// Scenario: Filtering employees where Salary > 50000
using (var context = new MyDbContext())
{
    // IEnumerable Approach (BAD for DB): Pulls ALL employees into server memory, then filters
    IEnumerable<Employee> empList = context.Employees;
    var filteredEnum = empList.Where(e => e.Salary > 50000).ToList();

    // IQueryable Approach (BEST for DB): Executes "SELECT ... WHERE Salary > 50000" inside SQL
    Server
    IQueryable<Employee> empQuery = context.Employees;
    var filteredQuery = empQuery.Where(e => e.Salary > 50000).ToList();
}
```

7. Explain Garbage Collection (GC) in .NET. What are Generations?

The .NET Garbage Collector manages automatic memory allocation and release. It targets the managed heap and runs in generations to optimize performance based on the principle that newly created objects tend to have shorter lifespans:

- **Generation 0:** Contains short-lived objects (e.g., local variables inside a method). GC runs here most frequently.
- **Generation 1:** Acts as a buffer zone between short-lived and long-lived objects. Objects surviving Gen 0 are promoted here.
- **Generation 2:** Contains long-lived objects (e.g., static data, application singletons). A Gen 2 collection is a full collection ("Full GC") and is highly resource-intensive.

8. What is the difference between Dispose() and Finalize()? How does the using statement work?

- **Dispose():** Belongs to the IDisposable interface. Used to explicitly release **unmanaged resources** (database connections, file streams, network sockets) immediately. It is invoked manually or via a using block.
- **Finalize():** A fallback destructor method called automatically by the Garbage Collector before an object is destroyed. It cannot be called explicitly.
- **using Statement:** Syntactic sugar for a try/finally block that guarantees Dispose() is executed even if an exception occurs inside the block.

```
using System.IO;

public class FileService
{
    public void WriteLog(string message)
    {
        // The using statement ensures the StreamWriter is disposed of properly
        using (StreamWriter writer = new StreamWriter("log.txt", true))
        {
            writer.WriteLine(message);
        } // writer.Dispose() is implicitly called here inside an invisible finally block
    }
}
```

9. Explain Extension Methods with a coding example.

Extension methods allow developers to add new methods to existing types without modifying their original source code, creating a derived type, or recompiling them. They must be defined inside a static class and must use the `this` keyword as the first parameter modifier.

```
public static class StringExtensions
{
    // Extends the built-in string class to count words
    public static int GetWordCount this string
    {
        if (string.IsNullOrEmpty(str))
            return 0;

        return str.Split(new[] { ' ', '\r', '\n' }, StringSplitOptions.RemoveEmptyEntries).Length;
    }
}

// Usage:
class Program
{
    static void Main
    {
        string text = "Hello world from .NET developer";
        int words = text.GetWordCount(); // Invoked like an instance method!
    }
}
```

10. What are Nullable Types, and how do you use the Null-Coalescing Operator (??) and Null-Conditional Operator (?.)?

- **Nullable Types (Nullable<T> or T?):** Allows value types (like `int`, `DateTime`) to represent an undefined or null state, which is vital when working with database fields that permit `NULL`.
- **?:** Checks if an object is null before accessing its properties/methods; returns null instead of throwing a `NullReferenceException`.

- **??**: Returns the left-hand operand if it isn't null; otherwise, returns the right-hand fallback value.

```
public class OperatorDemo
{
    public void ProcessUser
    {
        int? employeeAge = null; // Nullable value type

        // Null-coalescing operator: falls back to 18 if employeeAge is null
        int ageToUse = employeeAge ?? 18;

        Employee emp = null;
        // Null-conditional: safe access. If emp is null, department stays null instead of crashing
        string department = emp?.DepartmentName;

        // Chaining both together
        string finalDept = emp?.DepartmentName ?? "General Bench";
    }
}

public class Employee { public string DepartmentName { get; set; } }
```

Part 3: OOP Concepts (8 Questions)

1. What are the four core pillars of Object-Oriented Programming?

1. **Encapsulation:** Wrapping data (fields) and code (methods) together as a single unit while restricting direct access using access modifiers (private, protected).
2. **Abstraction:** Hiding complex internal implementation details and exposing only the essential interface to the user (via abstract classes and interfaces).
3. **Inheritance:** The mechanism by which a child class derives properties, behavior, and characteristics from a parent class to promote code reusability.
4. **Polymorphism:** The ability of an object to take on multiple forms (Compile-time via overloading; Runtime via overriding virtual/abstract methods).

2. Explain the difference between Method Overloading and Method

Overriding.

- **Method Overloading (Compile-time Polymorphism):** Multiple methods in the *same* class share the same name but have a different number or type of parameters (different signature).
- **Method Overriding (Runtime Polymorphism):** A subclass provides a new, specific implementation for a method that is already defined in its parent class using the virtual and override keywords.

```
public class Calculator
{
    // Overloading: Same name, different parameters
    public int Add(int a, int b) => a + b;
    public int Add(int a, int b, int c) => a + b + c;
}

public class ParentReport
{
    // Overriding: Base implementation
    public virtual void GenerateReport() => Console.WriteLine("Base PDF Report");
}

public class ExcelReport : ParentReport
{
    // Overriding: Redefining behavior at runtime
    public override void GenerateReport() => Console.WriteLine("Optimized MiniExcel Report");
}
```

3. What is the difference between an Abstract Class and an Interface? When should you use which?

- **Abstract Class:** Can contain both abstract methods (without bodies) and concrete methods (with bodies), fields, and access modifiers. It supports single inheritance only. **Use when** classes share a close identity and common implementation details ("Is-A" relationship).
- **Interface:** A pure contract containing only declarations of methods, properties, and

events (though modern C# permits default implementations). A class can implement multiple interfaces. **Use when** diverse, unrelated classes need to implement common standalone behaviors ("Can-Do" relationship).

```
public interface IRepository<T>
{
    void Add          ; // Pure contract
}

public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime CreatedDate { get; set; } // Shared structural property

    public void LogCreation => Console.WriteLine($"Created at {CreatedDate}");
}

// Derived class implements contract and inherits base structure
public class CustomerRepository : BaseEntity, IRepository<string>
{
    public void Add string    { /* implementation */ }
}
```

4. What is the purpose of the virtual, override, and sealed keywords?

- **virtual:** Marks a method inside a base class, indicating that it can be overridden in any inheriting subclasses.
- **override:** Expressly modifies or extends a base virtual or abstract method within a derived subclass.
- **sealed:** When applied to a class, it completely prevents inheritance. When applied to a method, it prevents subclasses from overriding that specific method further down the inheritance chain.

```

public class Logger
{
    public virtual void Log string    => Console.WriteLine(msg);
}

public sealed class EnterpriseLogger : Logger
{
    public override void Log string    => Console.WriteLine($"[Enterprise]: {msg}");
}

// Compilation Error! Cannot inherit from a sealed class.
// public class SubLogger : EnterpriseLogger { }

```

5. What are the differences between public, private, protected, internal, and protected internal access modifiers?

- **public:** Accessible from any code in the current assembly or any referencing assembly.
- **private:** Accessible only inside the specific class or struct containing it.
- **protected:** Accessible within the containing class and by any derived subclass instances.
- **internal:** Accessible strictly within the current assembly (.dll/.exe), but hidden from outside assemblies.
- **protected internal:** Accessible anywhere within the current assembly, *and* additionally inside inheriting subclasses located in other external assemblies.

6. Explain the concept of Deep Copy vs Shallow Copy.

- **Shallow Copy:** Copies only the high-level object structures. If an object contains reference types, the shallow copy copies the memory address pointer, meaning the original object and the copy share the exact same referenced sub-objects.
- **Deep Copy:** Creates a completely new replica of the parent object along with entirely new duplicates of all internal referenced sub-objects, ensuring complete isolation.

```

public class Department { public string Name { get; set; } }
public class Employee : ICloneable
{
    public int Id { get; set; }
}

```

```

public Department Dept { get; set; }

// Shallow Copy
public object Clone => this.MemberwiseClone();

// Deep Copy
public DeepCopy
{
    Employee copy = (Employee)this.MemberwiseClone();
    copy.Dept = new Department { Name = this.Dept.Name }; // Deep copy sub-object
    return copy;
}
}

```

7. What is the difference between a struct and a class?

- **class:** A reference type allocated on the Heap. It supports inheritance, destructors, and defaults to reference tracking.
- **struct:** A value type allocated on the Stack. It cannot inherit from other classes (but can implement interfaces). It offers better performance for lightweight objects with minimal fields that remain immutable after initialization.

8. Explain the SOLID Principles.

- **S - Single Responsibility:** A class should have one, and only one, reason to change.
- **O - Open/Closed:** Software entities should be open for extension, but closed for modification.
- **L - Liskov Substitution:** Subtypes must be completely substitutable for their base types without breaking application logic.
- **I - Interface Segregation:** Clients should not be forced to depend on massive interfaces containing methods they do not use. Break large interfaces into small, specific ones.
- **D - Dependency Inversion:** High-level modules should not depend on low-level modules; both must depend on abstractions (interfaces).

Part 4: ASP.NET Core & Web API (10 Questions)

1. Explain the ASP.NET Core Middleware Pipeline and how it differs from HTTP Modules/Handlers.

- **Middleware Pipeline:** Middleware components are C# classes assembled sequentially into an application pipeline to handle incoming HTTP requests and responses. Each component can execute logic before passing control via the next() delegate to the

subsequent middleware, or it can short-circuit the entire pipeline (e.g., Auth failure).

- **Difference:** Traditional IIS-tied HTTP Modules/Handlers were complex, configuration-heavy frameworks tied to IIS events. ASP.NET Core middleware is lightweight, fully customizable, modular, and works across cross-platform Kestrel engines.

```
// Inside Program.cs
var app = builder.Build();

// Order of execution is critical!
app.UseExceptionHandler("/error"); // 1. Catch exceptions
app.UseHttpsRedirection();         // 2. Enforce HTTPS
app.UseRouting();                  // 3. Match endpoints
app.UseAuthentication();           // 4. Identify who the user is
app.UseAuthorization();            // 5. Check user permissions

app.MapControllers();              // 6. Execute controller action
app.Run();
```

2. What are the three Service Lifetimes in ASP.NET Core Dependency Injection?

1. **Transient (AddTransient):** A brand new instance of the service is created every single time it is requested from the container. Best for lightweight, stateless services.
2. **Scoped (AddScoped):** A single instance is created per individual HTTP request cycle. It is shared across components processing that specific request. Best for database contexts (DbContext).
3. **Singleton (AddSingleton):** A single instance is created once when the application starts up and remains alive throughout the application's entire runtime lifecycle. Best for caching layers or configuration helpers.

3. What is the difference between AddMvc(), AddControllers(), and AddControllersWithViews()?

- **AddControllers():** Registers services required specifically for building RESTful backend **Web APIs** (handles controllers, routing, data annotations, validation, but strips away view engines).

- **AddControllersWithViews():** Configures services needed for standard **ASP.NET Core MVC** architectures utilizing traditional controllers paired with Razor view templates (.cshtml).
- **AddMvc():** Configures the entire MVC core framework suite including Razor Pages, Views, Web API capabilities, tag helpers, and localization engines. Avoid this in pure Web APIs to prevent unnecessary overhead.

4. How do you implement Model Validation in Web API, and how do you handle validation errors globally?

Model validation uses declarative Data Annotation attributes applied directly on the Data Transfer Object (DTO) properties. In modern ASP.NET Core, if a controller is marked with the [ApiController] attribute, the runtime automatically checks ModelState.IsValid and returns a 400 Bad Request containing details if validation fails.

```
using System.ComponentModel.DataAnnotations;

public class UserDto
{
    [Required(ErrorMessage = "Employee Name is mandatory.")]
    [StringLength(50, ErrorMessage = "Name cannot exceed 50 characters.")]
    public string Name { get; set; }

    [Required]
    [EmailAddress(ErrorMessage = "Invalid email format.")]
    public string Email { get; set; }

    [Range(1, 100, ErrorMessage = "Division selection must be between 1 and 100.")]
    public int DivisionId { get; set; }
}
```

5. Explain Routing in ASP.NET Core: Attribute Routing vs Conventional Routing.

- **Conventional Routing:** Defines centralized global routing templates inside Program.cs. Incoming requests match patterns like {controller}/{action}/{id?}. Mostly used in standard MVC applications with views.

- **Attribute Routing:** Maps URLs directly to individual action methods using [Route] and HTTP verb attributes ([HttpGet], [HttpPost]) at the controller level. **Highly preferred for Web APIs** to keep routes explicit and maintainable.

```
[ApiController]
[Route("api/[controller]")] // Global base route: api/attendance
public class AttendanceController : ControllerBase
{
    [HttpGet("summary/{employeeid}")] // Combined route: api/attendance/summary/300141
    public GetSummary int
    {
        return Ok(new { Empld = employeeid, Status = "Present" });
    }
}
```

6. How do you implement global exception handling in ASP.NET Core Web API?

Instead of wrapping every single controller method in noisy try-catch blocks, build a global exception handling middleware or use the `IExceptionHandler` interface (introduced in .NET 8). This catches unhandled exceptions globally and formats a clean response.

```
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging;
using System.Text.Json;

public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<GlobalExceptionMiddleware> _logger;
```



```

public GlobalExceptionHandlerMiddleware
{
    _next = next;
    _logger = logger;
}

public async InvokeAsync
{
    try
    {
        await _next(context); // Pass request forward
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "An unhandled exception occurred during execution.");
        await HandleExceptionAsync(context, ex);
    }
}

private static HandleExceptionAsync
{
    context.Response.ContentType = "application/json";
    context.Response.StatusCode = StatusCodes.Status500InternalServerError;

    var response = new {
        StatusCode = context.Response.StatusCode,
        Message = "Internal Server Error occurred. Please contact backend support."
    };

    return context.Response.WriteAsync(JsonSerializer.Serialize(response));
}
}

```

7. What is the difference between Action Results: IActionResult vs ActionResult<T>?

- **IActionResult:** An interface that allows returning any HTTP status code response format (OkObjectResult, NotFoundResult, BadRequestResult). However, it doesn't declare the exact return type, which can affect tool documentation like Swagger/OpenAPI.
- **ActionResult<T>:** Introduced to allow returning either a type deriving from IActionResult or the raw object type T directly. It preserves explicit type safety for API documentation

tools while retaining full HTTP status response flexibility.

```
[HttpGet("{id}")]
public GetEmployee int
{
    var emp = _repo.Find(id);
    if (emp == null)
        return NotFound(); // Returns an HTTP 404 ActionResult

    return emp; // Directly returns the Employee object type safely with HTTP 200 Ok
}
```

8. Explain JWT Authentication and how it works in a Web API environment.

JSON Web Token (JWT) is a secure, stateless mechanism for transmitting user claims as a JSON object.

1. The client logs in by sending valid credentials via a POST request.
2. The server verifies credentials, generates a cryptographically signed JWT containing specific claims (UserId, Roles), and sends it back to the client.
3. The client stores the token (e.g., in localStorage) and appends it to subsequent request headers as an Authorization: Bearer <token> header.
4. The API validates the cryptographic signature using its secret key; if valid, the request proceeds.

9. What is CORS, and how do you configure it in ASP.NET Core?

- **CORS (Cross-Origin Resource Sharing):** A browser-enforced security mechanism that prevents web pages from making requests to a domain different from the one that served the web page.
- **Configuration:** You must explicitly configure a CORS policy in the backend API to permit trusted origins (like an Angular front-end) to read the API responses.

```
// Program.cs
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAngularApp", policy => {
        policy.WithOrigins("http://localhost:4200") // Trust the specific origin
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();
app.UseCors("AllowAngularApp"); // Enforce before routing/controllers
```

10. What is Async/Await, and how does it prevent thread pool starvation?

The `async` and `await` keywords enable asynchronous programming. When an application encounters an I/O-bound operation (e.g., fetching a database query or a file network read), `await` yields control back to the Thread Pool, freeing up the current thread to process other concurrent incoming HTTP requests. Once the external database engine returns data, an available thread is picked up from the pool to complete the remaining code. This prevents **thread starvation** and allows the application to handle high volumes of concurrent requests.

Part 5: SQL Server (10 Questions)

1. What are Indexes in SQL Server? Explain the exact difference between Clustered and Non-Clustered Indexes.

Indexes speed up data retrieval operations from tables at the expense of extra storage and slower writes.

- **Clustered Index:** Physically reorders and stores the actual data rows of the table based on the indexed key columns. A table can have **only one** clustered index because data can only be physically sorted in one order. It is automatically created on Primary Keys by default.
- **Non-Clustered Index:** Contains a separate, isolated structure that holds index key pointers linked directly to the actual data rows (the clustered index key or row ID). A table can have **multiple** non-clustered indexes.

2. What is the difference between a Stored Procedure and a User-Defined Function (UDF)?

Feature	Stored Procedure (SP)	Function (UDF)
---------	-----------------------	----------------

Execution	Executed independently via EXEC.	Can be embedded directly inside SELECT, WHERE, JOIN.
DML Statements	Allowed to perform INSERT, UPDATE, DELETE operations.	Restricted to read-only SELECT operations.
Transaction Management	Fully supports BEGIN TRAN, COMMIT, ROLLBACK.	Not allowed within functions.
Return Value	Optional. Returns zero, one, or multiple result sets.	Must return a single value or a structured table.

3. Explain the difference between JOIN types: INNER, LEFT, RIGHT, and FULL OUTER.

- **INNER JOIN:** Returns only the matching rows present in both tables.
- **LEFT JOIN:** Returns all rows from the left table, along with matching rows from the right table. If no match exists, it appends NULL values for the right table columns.
- **RIGHT JOIN:** Returns all rows from the right table, paired with matching left rows (rarely used, as it can be rewritten as a LEFT JOIN).
- **FULL OUTER JOIN:** Combines both tables, returning rows when a match exists in either the left or right side. Unmatched sides are filled with NULL.

```
-- Selecting all employees and their assigned position mappings if they exist
SELECT E.EmpName, P.PositionTitle
FROM Tbl_Employee E
INNER JOIN Tbl_EmployeePositionMapping P ON E.EmpId = P.EmpId;
```

4. What are CTEs (Common Table Expressions)? Write a query demonstrating a CTE.

A CTE provides a temporary, named result set that exists solely within the execution scope of a

single SELECT, INSERT, UPDATE, or DELETE statement. It helps break down complex queries into readable, modular steps.

```
WITH AttendanceSummary_CTE AS (  
    SELECT  
        EmpId,  
        COUNT(CASE WHEN AttendanceStatus = 'Present' THEN 1 END) AS DaysPresent,  
        COUNT(CASE WHEN AttendanceStatus = 'Leave' THEN 1 END) AS DaysOnLeave  
    FROM Tbl_DailyAttendance  
    GROUP BY EmpId  
)  
-- Main outer query utilizing the CTE  
SELECT E.EmpName, A.DaysPresent, A.DaysOnLeave  
FROM Tbl_Employee E  
INNER JOIN AttendanceSummary_CTE A ON E.EmpId = A.EmpId  
WHERE A.DaysPresent < 20;
```

5. Explain Window Functions like ROW_NUMBER(), RANK(), and DENSE_RANK().

Window functions perform calculations across a set of table rows that are related to the current row, without collapsing them into a single row like a GROUP BY clause.

- **ROW_NUMBER():** Assigns a unique, sequential integer starting at 1 for each row within the partition group.
- **RANK():** Assigns values with gaps if duplicate sorting values exist (e.g., 1, 2, 2, 4).
- **DENSE_RANK():** Assigns sequential values without gaps for duplicate values (e.g., 1, 2, 2, 3).

```
SELECT EmpId, Salary, DepartmentId,  
    ROW_NUMBER() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC) as RowNum,  
    RANK() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC) as Rk,
```

```
DENSE_RANK() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC) as DenseRk
FROM Tbl_Employee;
```

6. What is the purpose of the COALESCE function? Provide an example.

COALESCE evaluates its arguments in order and returns the **first non-null value** in the list.

```
-- Real-world example: Check LeaveReason; if null, check RejectRemarks; if both null, output 'No
Remarks Given'
SELECT
    EmpId,
    COALESCE(LeaveReason, RejectRemarks, 'No Remarks Given') AS ManagementStatus
FROM Tbl_TourDetails;
```

7. How do you find the Nth Highest Salary in SQL Server? Provide the query.

Using DENSE_RANK() inside a CTE or subquery handles duplicate salaries reliably:

```
-- Finding the 3rd Highest Salary globally
WITH SalaryCTE AS (
    SELECT Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) as SalaryRank
    FROM Tbl_Employee
)
SELECT DISTINCT Salary
FROM SalaryCTE
WHERE SalaryRank = 3;
```

8. Explain database transactions and ACID Properties.

A Transaction is a single logical unit of database work. It must comply with ACID properties:

- **Atomicity:** All operations within the transaction succeed completely, or the entire transaction fails and rolls back completely ("All or nothing").
- **Consistency:** The database transitions from one valid, healthy state to another, maintaining all schema constraints and rules.
- **Isolation:** Concurrent execution of transactions leaves the database in the same state as if they were executed sequentially.
- **Durability:** Once a transaction commits, its changes survive permanently in the storage engine, even during sudden power outages or system crashes.

```
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Tbl_Accounts SET Balance = Balance + 500 WHERE AccountId = 1;
    UPDATE Tbl_Accounts SET Balance = Balance + 500 WHERE AccountId = 2;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION; -- Rolls back changes if an error occurs
    PRINT ERROR_MESSAGE();
END CATCH
```

9. What are SQL Server Performance Tuning steps you take for slow-running queries?

1. **Analyze Execution Plan:** Check for costly operations like **Table Scans** or **Index Scans** that point to missing indexes.
2. **Add Indexes:** Implement missing non-clustered indexes on columns frequently used inside WHERE, JOIN, and ORDER BY clauses.
3. **Avoid Wildcards at the Beginning:** Avoid syntax like LIKE '%text', which forces full table scans instead of utilizing index seeks.
4. **Use TRY_CONVERT Safely:** Avoid implicit type conversion warnings in conditions (e.g., comparing mismatched data types), as this stops the engine from utilizing index

structures.

5. **Update Statistics:** Run UPDATE STATISTICS regularly so the Query Optimizer can make accurate execution plans.

10. Explain TRY_CONVERT vs CONVERT and how to avoid conversion failures.

- **CONVERT:** Forces a type conversion. If the conversion fails (e.g., trying to parse the letter 'W' into a DATEDIFF calculation), the query crashes immediately.
- **TRY_CONVERT:** Attempts the conversion. If it fails, it returns NULL instead of crashing the query, allowing you to catch errors or fall back gracefully using COALESCE.

```
-- Safe execution query using TRY_CONVERT to handle invalid strings safely
SELECT
    TourId,
    COALESCE(TRY_CONVERT(INT, DurationString), 0) AS CalculatedDays
FROM Tbl_TourDetails;
```

Part 6: Git & GitLab (8 Questions)

1. What is the difference between git fetch and git pull?

- **git fetch:** Downloads the latest commits, branches, and tags from the remote repository (GitLab) to your local tracking system. It **does not modify or merge changes** into your active working directory.
- **git pull:** A combined command that executes git fetch first, then automatically performs a git merge to apply those remote commits directly into your current working branch.

2. How do you resolve a Merge Conflict? What are the exact steps?

A merge conflict occurs when two developers modify the exact same lines of code in the same file on different branches, and Git cannot determine which version to use.

1. Identify the conflicted files using git status.
2. Open the conflicted files. You will see markers like:

```
<<<<<<< HEAD
Current Local Changes (My Code)
=====
```


Incoming Changes from Remote
>>>>>> feature-branch

3. Manually edit the file to select the correct code and remove the conflict markers (<<<<<<, =====, >>>>>>).
4. Save the file, run `git add <filename>`, and then commit the resolution using `git commit -m "Resolved merge conflict"`.

3. What is the difference between git merge and git rebase?

- **git merge:** Combines the source branch into the target branch by creating a dedicated **Merge Commit**. This preserves the chronological history of how branches branched off and merged back in.
- **git rebase:** Rewrites the project history by taking all new commits from your current branch and reapplying ("moving") them sequentially onto the tip of the target branch. This results in a clean, linear git history without extra merge commits.

4. What is git stash, and when should you use it?

`git stash` temporarily shelves (saves) uncommitted modifications (staged or unstaged) so you can work on a completely different task without losing your progress. You can later restore those changes to your working branch.

```
# Stash active changes away safely
git stash
```

```
# Switch to main branch to fix an urgent bug
git checkout main
# ... fix bug and push ...
```

```
# Return to feature branch and restore stashed work
git checkout feature-branch
git stash pop
```

5. What is a Merge Request (MR) or Pull Request (PR) workflow in GitLab?

1. Pull the latest code from main or develop.
2. Create a separate feature branch locally: `git checkout -b feature/user-modal`.

3. Develop your features, write code, commit, and push the branch to GitLab.
4. Log into GitLab and open a **Merge Request** targeting the develop or main branch.
5. The MR runs automated CI pipelines (builds, tests, checks rules) and allows team leads to perform a code review.
6. Once approved, it is merged into the target branch.

6. What is git cherry-pick, and when should it be used?

git cherry-pick applies the changes introduced by a specific, single commit from another existing branch and applies it as a brand new commit onto your current active branch. This is useful when you want to copy a specific bug fix from an experimental branch into your release branch without merging the entire branch history.

```
# Find the commit ID from another branch, switch to your target branch, and apply it
git checkout release-branch
git cherry-pick a1b2c3d4
```

7. How do you undo the last commit that has already been pushed to a remote repository?

- **Safe Approach (git revert):** Creates a brand new commit that does the exact opposite of the target commit, reversing the code changes safely without modifying the existing commit history.
- **Destructive Approach (git reset --hard):** Deletes the commit history entirely. This requires a forced push (git push -f) and should be avoided on shared team branches.

```
# Recommended for production: Reverts commit and records the action clearly
git revert <commit-id>
git push origin main
```

8. Explain Git tagging and its purpose in release management.

Git tags mark specific points in the repository's history as important milestones, typically used to track release versions (e.g., v1.0.0, v2.1.0). Unlike branches, tags do not change over time. In GitLab, pushing a release tag often triggers automated deployment pipelines to production servers.

Part 7: Project Structure & Architecture (8 Questions)

1. What is Clean Architecture or 3-Tier Architecture? Explain the separation of layers.

Clean Architecture emphasizes separating the core business logic from external frameworks, databases, and UI concerns. It typically organizes code into independent layers:

- **Domain (Core) Layer:** The center of the application. Contains enterprise entities, value objects, and core business rules. It has **zero dependencies** on other layers.
- **Application Layer:** Contains application use cases, business interfaces, and workflow logic (e.g., MediatR handlers). It depends *only* on the Domain layer.
- **Infrastructure Layer:** Implements interfaces defined in the application layer. Handles data access (EF Core, DbContext), file systems, or external API communications.
- **Presentation (API) Layer:** Exposes endpoints to clients (Controllers, Minimal APIs). It depends on Application and Infrastructure to bootstrap dependency injections.

2. What is the Repository Pattern, and why is it used?

The Repository Pattern acts as a buffer layer between the core business logic and the underlying data access abstraction layer. It exposes collection-like interfaces (Add, Get, Remove) to the application layer, decoupling it from the data source.

- **Advantages:** It isolates database-specific querying technologies (like EF Core or Dapper), making the application easier to maintain and test via mocking frameworks.

3. What is the difference between DTOs (Data Transfer Objects) and Database Entities?

- **Database Entities:** C# classes that match the exact database table schemas (e.g., fields mapped via Entity Framework). They are used by data access layers.
- **DTOs:** Plain C# objects designed specifically for moving optimized data payloads between the backend API endpoints and the front-end clients (e.g., Angular). They exclude internal sensitive data fields (like passwords) and prevent cyclic object referencing errors during serialization.

4. Explain the CQRS Pattern (Command Query Responsibility Segregation).

CQRS physically separates application read operations from modification operations:

- **Commands:** Change the state of the application or database (Create, Update, Delete). They do not return data, other than confirmation IDs or status codes.
- **Queries:** Retrieve data from the system without modifying anything (GetById, GetAll). Queries can be routed to optimized database read replicas or run through performance frameworks like Dapper.

5. What is the Unit of Work Pattern, and how does it complement the Repository Pattern?

The Unit of Work pattern maintains a centralized list of business operations affected by a business transaction. It coordinates writing out changes by ensuring that multiple repositories share a single database context transaction. In .NET, Entity Framework Core's DbContext implicitly acts as both the Repository and the Unit of Work pattern behind the scenes, committing changes collectively when SaveChanges() is invoked.

6. What is the difference between Monolithic and Microservices Architecture?

- **Monolith:** The entire application (API, UI, Data access) is compiled and deployed together as a single unit. It is easy to build, test, and deploy early on, but can become difficult to scale or maintain as the codebase grows.
- **Microservices:** Breaks the application down into independent services organized around specific business capabilities (e.g., Attendance Service, Billing Service). Each service owns its own database and communicates via lightweight protocols (HTTP/REST or message brokers). This improves scalability but adds operational and network complexity.

7. How do you manage application configurations safely across different deployment environments?

Use environment-specific configuration files (appsettings.json, appsettings.Development.json, appsettings.Production.json). The .NET runtime automatically overrides settings based on the ASPNETCORE_ENVIRONMENT variable. For production environments, sensitive keys should be injected securely via system Environment Variables or cloud key vaults, rather than hardcoded in the source code.

8. What is the purpose of AutoMapper in a project, and what are its pros and cons?

AutoMapper automatically maps values from one object type to another (e.g., converting a Customer database entity into a CustomerDto object) based on matching property names.

- **Pros:** Eliminates boilerplate mapping code (dto.Name = entity.Name).
- **Cons:** Can make debugging difficult if mapping configurations are misconfigured, and can introduce slight performance overhead compared to explicit manual mapping using Select expressions.

Part 8: Live Coding & Logical Questions

(10 Questions)

1. Reverse a string without using built-in methods.

```
public class StringLogic
{
    public string Reverse string
    {
        if (string.IsNullOrEmpty(input)) return input;

        char[] chars = input.ToCharArray();
        int left = 0;
        int right = input.Length - 1;

        while (left < right)
        {
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;
            left++;
            right--;
        }
        return new string(chars);
    }
}
```

2. Check if a given string is a Palindrome.

```

public bool IsPalindrome string
{
    if (string.IsNullOrEmpty(input)) return false;

    // Normalize string: convert to lowercase and remove spaces if needed
    string cleanStr = input.Replace(" ", "").ToLower();
    int min = 0;
    int max = cleanStr.Length - 1;

    while (min < max)
    {
        if (cleanStr[min] != cleanStr[max])
            return false;
        min++;
        max--;
    }
    return true;
}

```

3. Count occurrences of each character in a string using LINQ.

```

using System;
using System.Linq;

public void CharacterCount string
{
    if (string.IsNullOrEmpty(text)) return;

    var counts = text.GroupBy(c => c)
        .Select(g => new { Char = g.Key, Count = g.Count() });

    foreach (var item in counts)
    {
        Console.WriteLine($"Character '{item.Char}': {item.Count} times");
    }
}

```

```
}  
}
```

4. Find duplicate elements in an integer array using LINQ.

```
using System.Collections.Generic;  
using System.Linq;  
  
public int FindDuplicates(int[] numbers)  
{  
    return numbers.GroupBy(n => n)  
        .Where(g => g.Count() > 1)  
        .Select(g => g.Key)  
        .ToList();  
}
```

5. Check if two strings are Anagrams (e.g., listen and silent).

```
using System;  
using System.Linq;  
  
public bool AreAnagrams(string str1, string str2)  
{  
    if (str1.Length != str2.Length) return false;  
  
    // Sort both character arrays and compare their sequence  
    return str1.ToLower().OrderBy(c => c).SequenceEqual(str2.ToLower().OrderBy(c => c));  
}
```

6. Implement the FizzBuzz problem using a loop.

```
public void FizzBuzz(int limit)
{
    for (int i = 1; i <= limit; i++)
    {
        if (i % 3 == 0 && i % 5 == 0)
            Console.WriteLine("FizzBuzz");
        else if (i % 3 == 0)
            Console.WriteLine("Fizz");
        else if (i % 5 == 0)
            Console.WriteLine("Buzz");
        else
            Console.WriteLine(i);
    }
}
```

7. Find the second largest number in an integer array without sorting.

```
public int GetSecondLargest(int[] arr)
{
    int largest = int.MinValue;
    int secondLargest = int.MinValue;

    foreach (int num in arr)
    {
        if (num > largest)
        {
            secondLargest = largest;
            largest = num;
        }
        else if (num > secondLargest)
        {
            secondLargest = num;
        }
    }
}
```



```

        largest = num;
    }
    else if (num > secondLargest && num != largest)
    {
        secondLargest = num;
    }
}
return secondLargest;
}

```

8. Remove duplicate characters from a string.

```

using System;
using System.Collections.Generic;

public string RemoveDuplicateChars string
{
    if (string.IsNullOrEmpty(input)) return input;

    HashSet<char> uniqueChars = new HashSet<char>();
    List<char> result = new List<char>();

    foreach (char c in input)
    {
        if (uniqueChars.Add(c)) // Returns true if element was successfully added
        {
            result.Add(c);
        }
    }
    return new string(result.ToArray());
}

```

9. Generate Fibonacci series up to N numbers.

```

public void PrintFibonacci(int n)
{
    int a = 0, b = 1, c;
    if (n >= 1) Console.Write(a + " ");
    if (n >= 2) Console.Write(b + " ");

    for (int i = 2; i < n; i++)
    {
        c = a + b;
        Console.Write(c + " ");
        a = b;
        b = c;
    }
}

```

10. Check if a number is Prime.

```

public bool IsPrime(int number)
{
    if (number <= 1) return false;
    if (number == 2) return true;
    if (number % 2 == 0) return false;

    var boundary = (int) Math.Floor(Math.Sqrt(number));

    for (int i = 3; i <= boundary; i += 2)
    {
        if (number % i == 0) return false;
    }
}

```

```
    return true;
}
```

Part 9: Scenario-Based Questions (10 Questions)

1. Scenario: Your API is running slow when downloading massive Excel reports containing thousands of records. How do you resolve this speed issue?

Solution Strategy:

Traditional tools like EPPlus or standard ClosedXML load the entire dataset into memory to build the DOM, which can cause out-of-memory errors and server crashes.

Resolution: Implement **MiniExcel** logic, which uses streaming architectures to read and write rows immediately, keeping the memory footprint minimal and improving execution speed.

```
using System.IO;
using Microsoft.AspNetCore.Mvc;
using MiniExcelLibs;

[HttpGet("download-report")]
public async DownloadLargeReport
{
    var memoryStream = new MemoryStream();

    // Fetch records lazily as an IQueryable stream
    var largeDataList = _context.EmployeeOutletEfficiencyReport.AsNoTracking().ToList();

    // MiniExcel stream-saves records directly to saving buffer memory
    await memoryStream.SaveAsync(largeDataList);
    memoryStream.Seek(0, SeekOrigin.Begin);

    return File(memoryStream,
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet", "EfficiencyReport.xlsx");
}
```

2. Scenario: Your system logs multiple duplicate rows for an employee's "Attendance Summary" on the same day due to mobile app re-sync events. How do you handle it cleanly in code?

Solution Strategy:

Implement an explicit priority hierarchy in the business engine to evaluate competing records (e.g., favoring actual field events like "Market Visit" over system closures like "End Day") to ensure consistent reporting data.

```
public string DeduplicateAgenda      string
{
    if (logs == null || !logs.Any()) return "Absent";

    // Priority Rule Engine: actual business activity takes priority over generic tracking records
    if (logs.Contains("Market Visit")) return "Market Visit";
    if (logs.Contains("Office Attendance")) return "Office Attendance";
    if (logs.Contains("End Day")) return "End Day";

    return logs.First();
}
```

3. Scenario: A user opens a modal form, selects an item from a new dropdown field, but saving fails because the database field requires a primary key link. How do you implement the frontend-backend synchronization safely?

Solution Strategy:

Implement proper validation and error handling on both ends: validate the selection in the UI modal, and check the values against database constraints in the API controller before saving.

```

[HttpPost("save-division")]
public async SaveCustomerDivision
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState); // Returns explicit UI notifications

    // Verify foreign key integrity against the database
    bool divisionExists = await _context.Divisions.AnyAsync(d => d.Id == model.DivisionId);
    if (!divisionExists)
        return BadRequest(new { Message = "The selected Division is invalid or missing." });

    // Execute save operation...
    return Ok(new { Success = true });
}

```

4. Scenario: A reporting manager dashboard needs to display if any hierarchical direct reports are currently marked "On Leave". How do you write this check inside an existing user query?

Solution Strategy:

Join the employee table with the active leave tracking tables using the team lead's identifier to check the team's live status.

```

public async Task<List<EmployeeStatusDto>> GetSubordinatesStatusAsync(int managerId)
{
    return await _context.Employees
        .Where(e => e.ManagerId == managerId)
        .Select(e => new EmployeeStatusDto
        {
            EmployeeId = e.Id,
            EmployeeName = e.Name,
            // Check if active leave entry exists for today's date
            IsOnLeave = _context.TourDetails.Any(t => t.EmplId == e.Id && t.Date ==
                DateTime.Today && t.Status == "Approved")
        })
}

```

```
    })  
    .ToListAsync();  
}
```

5. Scenario: The legal department requires adding "RejectRemarks" and "LeaveReason" auditing metrics onto an active table. How do you push these schema updates safely without breaking production systems?

Solution Strategy:

Add the new column fields as nullable (NULL), ensuring that existing production applications can continue running without breaking queries while the new logic is being deployed.

```
-- Step 1: Execute schema adjustment scripts  
ALTER TABLE Tbl_TourDetails  
ADD RejectRemarks NVARCHAR(500) NULL,  
    LeaveReason NVARCHAR(500) NULL;
```

```
// Step 2: Update the backend entity model to reflect the new nullable database fields  
public class TourDetail  
{  
    public int Id { get; set; }  
    public string? RejectRemarks { get; set; } // Nullable string maps cleanly to NULL column  
    public string? LeaveReason { get; set; }  
}
```

6. Scenario: Your API needs to consume a third-party microservice, but that microservice occasionally suffers brief network drops. How

do you prevent your API from failing?

Solution Strategy:

Use an HTTP resilience library like **Polly** to implement a retry mechanism. This automatically retries the request a specified number of times with an exponential delay before returning a failure to the user.

```
// Configure Polly inside Program.cs
builder.Services.AddHttpClient("ExternalClient")
    .AddTransientHttpErrorPolicy(policy =>
        policy.WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
            retryAttempt))));
```

7. Scenario: Two users read a row with a balance of \$100 simultaneously. Both attempt to update the balance at the same time. How do you prevent them from overwriting each other's changes?

Solution Strategy:

Implement **Optimistic Concurrency Control**. Add a tracking column (like a RowVersion timestamp) to the table. When updating, check if the version matches the version that was originally read; if it has changed, reject the update.

```
public class Account
{
    public int Id { get; set; }
    public decimal Balance { get; set; }

    [Timestamp] // Concurrency token handled by EF Core
    public byte[] RowVersion { get; set; }
}
```

```
// In the repository layer:
try {
    _context.SaveChanges();
}
catch (DbUpdateConcurrencyException ex) {
    // Catch the race condition early and notify the user to reload the fresh data
}
```

8. Scenario: You are tasked with migrating a legacy API to an asynchronous architecture, but you cannot change the signatures of the entry methods immediately. How do you avoid deadlocks?

Solution Strategy:

Mixing synchronous and asynchronous code (like using `.Result` or `.Wait()`) can cause thread deadlocks in older ASP.NET runtime engines. The safest long-term approach is to update the application asynchronously from top to bottom. If you must call async code from a synchronous block during a migration, use explicit task run strategies:

```
// Safe temporary execution wrapper
public string GetLegacyDataSync(int id)
{
    return Task.Run(async () => await MyAsyncApiCall(id)).GetAwaiter().GetResult();
}
```

9. Scenario: A client complains that your application's search feature is slow when filtering by creation date and user ID. The underlying database table contains millions of rows. How do you resolve this?

Solution Strategy:

Create a composite non-clustered index on the columns used for filtering (`UserId` and `CreationDate`), allowing the database engine to locate the matching records without performing a full table scan.


```
CREATE NONCLUSTERED INDEX IX_Tbl_Orders_UserId_CreationDate
ON Tbl_Orders (UserId, CreationDate)
INCLUDE (OrderTotal, Status); -- Include extra display fields to prevent lookup steps
```

10. Scenario: An application bug allows users to submit text comments containing HTML tag blocks, creating a vulnerability to Cross-Site Scripting (XSS) attacks. How do you protect your endpoints?

Solution Strategy:

Use data validation tools and libraries like HtmlSanitizer to clean up text content or strip out dangerous script blocks before saving the input to the database.

```
using Ganss.Xss;

public string SecureCommentInput string
{
    var sanitizer = new HtmlSanitizer();
    // Strips out dangerous HTML tags like <script> while preserving plain text
    return sanitizer.Sanitize(rawUserInput);
}
```

Part 10: Salary, Notice Period & Closing (8 Questions)

1. What are your salary expectations?

Answer: > "My current CTC is [Your Current CTC]. Based on my two years of hands-on experience in backend development, .NET Core optimization, and database design, along with

the technical responsibilities of this role, I am expecting a salary in line with industry standards—around [Your Expected CTC or % hike, e.g., 30-40% hike]. I am open to a reasonable discussion once we conclude the technical rounds."

2. What is your notice period? Can it be negotiated?

Answer: > "My official notice period is [e.g., 60 or 90 days]. However, I have built a solid documentation plan for my active features, so I can speak with my manager about an early release if a firm offer is extended. I estimate we could potentially shorten it to [e.g., 30 or 45 days]."

3. If you get a counter-offer from your current company, will you stay?

Answer: > "No, I will not change my decision. My reason for seeking a change is to expand my technical scope, work on larger-scale architectures, and take on new challenges. A counter-offer might change the financial compensation temporarily, but it won't change the type of work or technology stack available to me. I am fully committed to transitioning to a new environment."

4. Are you comfortable relocating, or do you have any location preferences?

Answer: > "My primary preference is targeting roles in **Noida**, as it is an excellent tech hub for my career growth and logistically convenient for me. I am fully comfortable working out of an office in the Noida region."

5. Why should we hire you over other candidates?

Answer: > "While many candidates can write functional C# code, I bring strong hands-on experience in query profiling, performance tuning, and optimizing application layers (like my work with MiniExcel). I have a proven track record of delivering clean code under tight deadlines, working effectively with senior leads, and taking ownership of features from start to finish. I am ready to be productive in your stack immediately."

6. What will you do if your current manager asks you to stop your resignation process?

Answer: > "I will explain to my manager respectfully that my decision is driven by a desire for professional growth and to gain experience with new architectures. I will reassure them that I am fully committed to ensuring a smooth knowledge transfer and documenting all my active tasks before my final day."

7. Do you have any active job offers right now?

- *If you do not:* "I am actively interviewing with a couple of select companies in Noida, and I am currently in advanced technical stages, but I haven't accepted any offers yet. Your opportunity aligns closely with my skills."
- *If you do:* "Yes, I have an active offer in hand, but I am exploring this opportunity because

your company's technology stack and engineering culture align better with my long-term career goals."